7.4 图的其他运算

1. 求图的生成树
2. 求最小生成树
3. 求最短路径
4. 求关节点和重连通分量（略）
5. 拓扑排序
6. 求关键路径

3. 求最短路径（不考）

典型用途：交通问题。如：城市A到城市B有多条线路，但每条线路的交通费（或所需时间）不同，那么，如何选择一条线路，使总费用（或总时间）最少?

问题抽象：在带权有向图中A点（源点）到达B点（终点）的多条路径中，寻找一条各边权值之和最小的路径，即最短路径。

（注：最短路径与最小生成树不同，路径上不一定包含n个顶点）

两种常见的最短路径问题：

一、 单源最短路径—用Dijkstra（迪杰斯特拉）算法

二、所有顶点间的最短路径—用Floyd（弗洛伊德）算法

一顶点到其余各顶点（$v_0 \rightarrow j$）

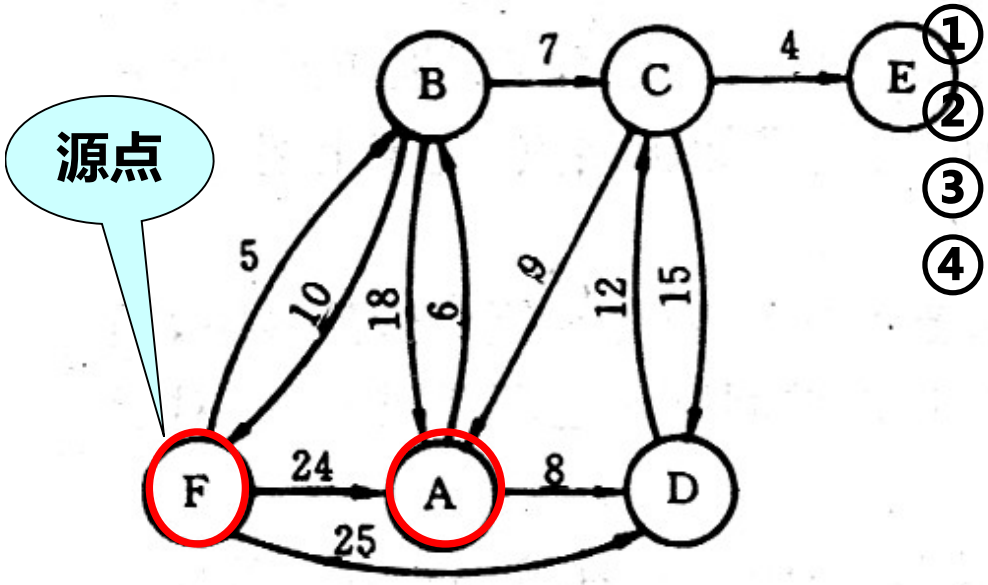
任意两顶点之间（$i \rightarrow j$）

一、单源最短路径（Dijkstra算法）

一顶点到其余各顶点（$v_0 \to j$）

•目的： 设一有向图G=（V, E），已知各边的权值，以某指定点v_0为源点，求从v_0到图的其余各点的最短路径。限定各边上的权值大于或等于0。

例1：



从F→A的路径有4条：

① F→A： 24

② F→B→A： 5＋18=23

③ F→B→C→A： 5+7+9=21

④ F→D→C→A： 25+12+9=36

想一想：

从F→B的最短路径是哪条？

从F→C的最短路径是哪条？

如何自动求出最短路径？按路径“长度” 递增的次序逐步产生

二、所有顶点之间的最短路径

可以通过调用n次Dijkstra算法来完成，
时间复杂度为$O(n^3)$

还有更简单的一个算法：Floyd算法（自学）

5.拓扑排序

问题:

假设以有向图表示一个工程的施工图或程序的数据流图，则图中不允许出现回路。

检查有向图中是否存在回路的方法之一，是对有向图进行**拓扑排序**。

何谓“拓扑排序”？

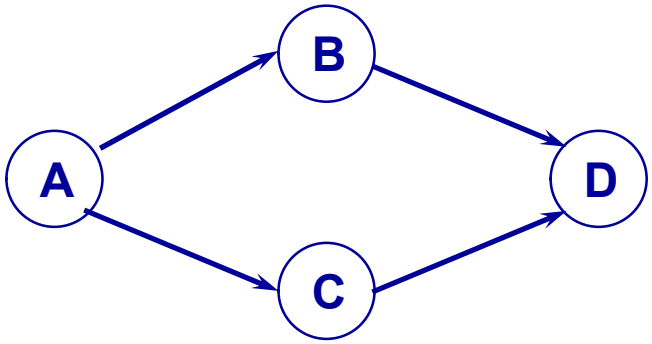
对有向图进行如下操作：

按照有向图给出的次序关系，将图中顶点排成一个线性序列，对于有向图中没有限定次序关系的顶点，则可以人为加上任意的次序关系。

由此所得顶点的线性序列称之为

拓扑有序序列

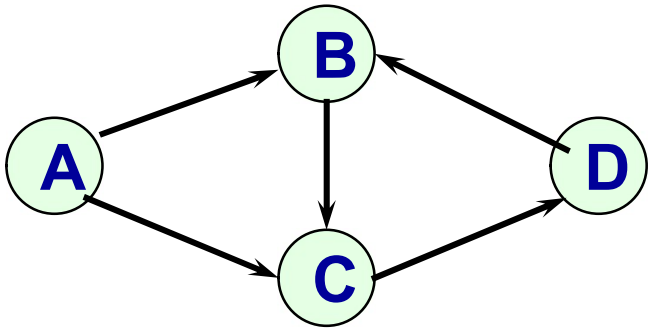
例如：对于下列有向图



可求得拓扑有序序列：

A B C D　或　A C B D

反之，对于下列有向图



不能求得它的拓扑有序序列。

因为图中存在一个回路 {B, C, D}

如何进行拓扑排序？

一、从有向图中选取一个**没有前驱**的顶点，并输出之；

二、从有向图中**删去此顶点以及所有以它为尾的弧**；

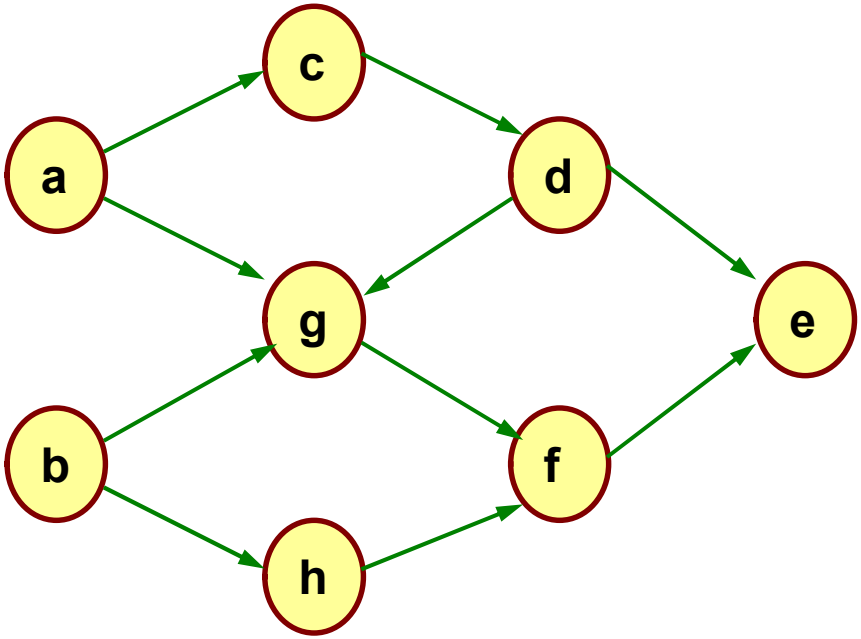
重复上述两步，直至图空，或者图不空但找不到无前驱的顶点为止。

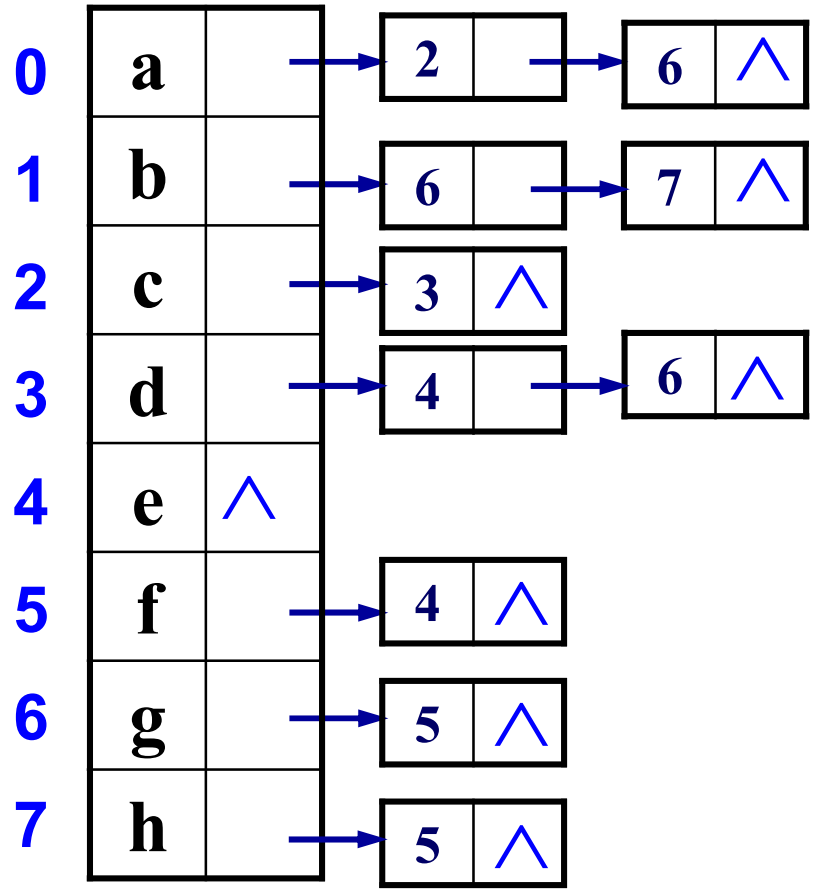
a b h c d g f e

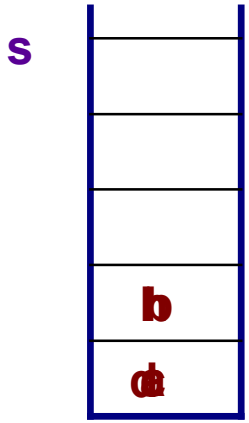
在算法中需要用定量的描述替代定性的概念

没有前驱的顶点 == 入度为零的顶点

删除顶点及以它为尾的弧 == 弧头顶点的入度减1







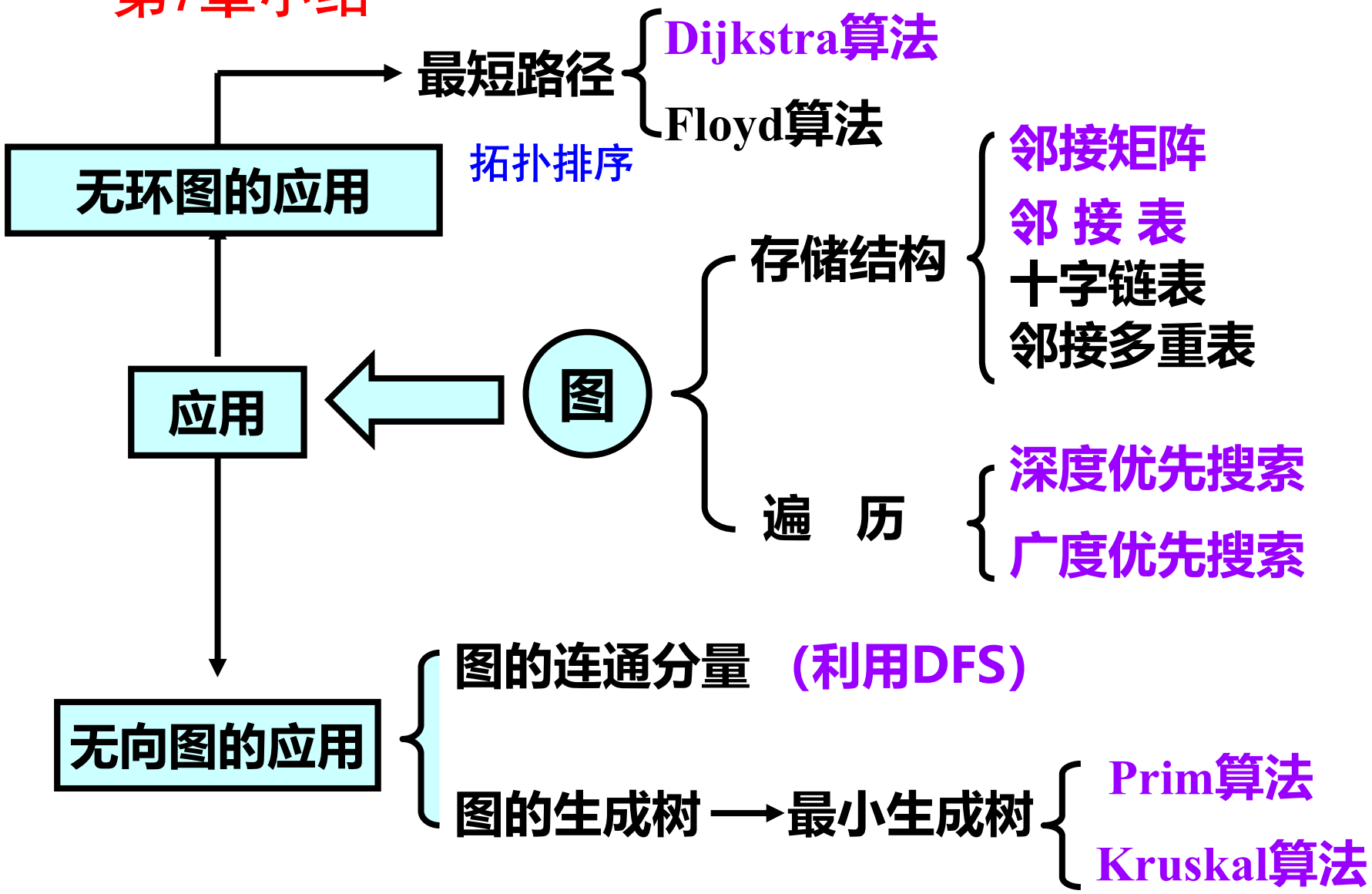
	a	b	c	d	e	f	g	h
indegree	0	0	0	0	0	0	0	0

输出次序：b h a c d g f e

算法描述

```
Status Topologicalsort(ALGraph  G) {
 FindinDegree(G,indegree);
 InitStack(s)
 For(i=0;i<G.vexnum;++i)
     if(!indegree[i])   push(s,i);
 Count=0;
 While(!StackEmpty(s)) {
    pop(s,i);  printf(i,G.vertices[i].data);  ++count;
    for(p=G.vertices[i].firstarc;p;p=p->nextarc) {
        k=p->adjvex;
        if(!(- -indegree[k]))   push(s,k);  }      }
 if(count<g.vexnum)   return ERROR;
 }
```

第7章小结


最短路径
Dijkstra算法
Floyd算法
无环图的应用
拓扑排序
应用
图
存储结构
邻接矩阵
邻 接 表
十字链表
邻接多重表
遍 历
深度优先搜索
广度优先搜索
无向图的应用
图的连通分量 (利用DFS)
图的生成树 ⟶ 最小生成树
Prim算法
Kruskal算法